

Kubernetes Phase 2

Complete Blueprint

From Concepts to Production Building Blocks

What this phase builds on:

Phase 1 gave you the conceptual foundation — Pods, Services, Deployments, ConfigMaps, Namespaces, `kubectl` basics. You can now READ a manifest and explain what each line does.

What Phase 2 will give you:

The production muscles. By the end of this phase you will know how to keep apps healthy, give them durable storage, isolate them with security, auto-scale them under load, and run stateful workloads like databases. These are the patterns you will see in every real-world Kubernetes setup.

How to use this document: Read each topic in order. Each section has the *concept*, the *YAML*, and the *kubectl* commands. Don't just read — type every command in your kind cluster as you go.

Table of Contents

1. Probes — Liveness, Readiness, Startup
2. Resource Management — Requests, Limits, QoS
3. Persistent Storage — PV, PVC, StorageClass
4. StatefulSets — Stateful Workloads
5. Jobs & CronJobs — One-Off and Scheduled Tasks
6. DaemonSets — One Pod Per Node
7. Init Containers & Sidecar Pattern
8. Advanced Ingress — TLS, Path Routing, Controllers
9. Horizontal Pod Autoscaler (HPA)
10. RBAC & ServiceAccounts
11. NetworkPolicies — Pod-Level Firewall
12. Pod Scheduling — nodeSelector, Affinity, Taints
13. Advanced kubectl — jsonpath, custom-columns
14. Phase 2 Project — 3-Tier App on K8s
15. Phase 2 Summary Cheatsheet

1. Probes — Liveness, Readiness, Startup

Teaching Kubernetes how to check if your app is healthy

The Problem

Kubernetes can tell whether a container is RUNNING. It cannot tell whether your app inside is actually healthy. Your app might be deadlocked, stuck in a boot loop, or returning errors — but the container process is still alive, so K8s thinks everything is fine.

Probes solve this. They are health checks Kubernetes runs against your container to make smart decisions: should I restart this pod? Should I send it traffic? Is it still starting up?

The Three Types of Probes

Probe	Question It Answers	What K8s Does If It Fails
Liveness	Is my app alive?	Restart the container
Readiness	Is my app ready for traffic?	Remove pod from Service endpoints
Startup	Has my app finished starting?	Suppress liveness/readiness checks until it passes

Liveness Probe — The Watchdog

If the liveness probe fails repeatedly, Kubernetes assumes your container is broken and restarts it. Use this for catching deadlocks and frozen states.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15  # wait 15s before first check
  periodSeconds: 10       # check every 10s
  failureThreshold: 3     # 3 fails = restart
```

Readiness Probe — The Traffic Director

If the readiness probe fails, K8s removes the pod from the Service's endpoint list — no traffic is sent to it. The pod is NOT restarted. Use this for warmup periods, dependency checks (DB connection), or temporary load.

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

Startup Probe — Slow Boot Protector

Some apps take a long time to start (Java apps, ML model loaders). The startup probe tells K8s 'don't run liveness/readiness yet — wait until I'm done starting.' Without it, slow apps get killed by liveness probes during boot.

```
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30      # allow 30 * 10s = 5 minutes to start
  periodSeconds: 10
```

Probe Methods — How K8s Checks

Method	How It Works	When to Use
httpGet	GETs a URL, expects 200-399	Web apps, REST APIs (most common)
tcpSocket	Opens a TCP connection	Databases, message queues
exec	Runs a command inside container	Custom logic, file checks

Real-World Pattern (All Three Probes)

```
spec:
  containers:
  - name: my-app
    image: myapp:v1
    ports:
    - containerPort: 8080
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      failureThreshold: 30
      periodSeconds: 10
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }
      periodSeconds: 5
```

Common pitfall: A pod stuck in CrashLoopBackOff with rapid restarts almost always has a liveness probe firing before the app is ready. Add a startupProbe — it suppresses liveness checks during slow startup.

2. Resource Management — Requests, Limits, QoS

Telling Kubernetes how much CPU and memory your app needs

The Problem

Without resource declarations, the K8s scheduler places pods randomly on any node. One memory-hungry pod can starve every other pod on its node. Worse, the OS might OOM-kill (out-of-memory kill) your apps without warning.

Requests vs Limits

	Requests	Limits
Purpose	Minimum guaranteed resources	Maximum allowed resources
Used by	The scheduler (for placement)	The runtime (kubelet enforces)
If exceeded?	N/A — it's a guarantee	CPU is throttled, memory triggers OOMKill
Analogy	'I need at least this room'	'Don't let me use more than this'

Units You Need to Know

CPU: measured in cores or millicores. 1 = 1 full CPU core. 500m = 0.5 cores (millicores). 100m = 0.1 cores. Most small apps need 100m–500m.

Memory: measured in bytes. Mi = mebibytes (1024^2 bytes), Gi = gibibytes (1024^3 bytes). Use Mi/Gi, not MB/GB — they differ slightly and Mi/Gi is the K8s standard.

Real Example

```
spec:
  containers:
  - name: my-app
    image: nginx
    resources:
      requests:
        cpu: "200m"      # I need at least 0.2 CPU
        memory: "256Mi"  # I need at least 256 MiB RAM
      limits:
        cpu: "500m"      # Don't let me use more than 0.5 CPU
        memory: "512Mi"  # Kill me if I go past 512 MiB
```

QoS Classes — How K8s Decides Who Dies First

When a node runs out of memory, Kubernetes evicts pods. It picks who to evict based on QoS (Quality of Service) class — assigned automatically based on your requests/limits.

QoS Class	When Assigned	Eviction Order
Guaranteed	requests = limits for ALL containers	Last to be evicted (safest)
Burstable	requests < limits, or only requests set	Evicted second
BestEffort	No requests or limits set at all	Evicted first (most at risk)

Production rule: Always set requests on every container. Without them, the scheduler places pods blindly and you get random crashes under load. Set limits on memory always; on CPU only if you really want throttling.

Useful Commands

```
# See node resource usage
kubectl top nodes
kubectl top pods

# See what each node has allocated
kubectl describe node <node-name> | grep -A 10 "Allocated resources"

# Check QoS class of a pod
kubectl get pod <pod> -o jsonpath='{.status.qosClass}'

# Find pods without resource limits (bad practice detector)
kubectl get pods -A -o json | jq '.items[] |
  select(.spec.containers[].resources.limits == null) |
  .metadata.name'
```

3. Persistent Storage — PV, PVC, StorageClass

Giving pods storage that survives restarts and reschedules

The Problem

By default, when a pod restarts or moves to another node, its filesystem is **destroyed**. This is fine for stateless apps (web servers) but disastrous for databases, file uploads, or any data you want to keep.

Kubernetes solves this with three abstractions: **PersistentVolume**, **PersistentVolumeClaim**, and **StorageClass**.

The Three Layers

Object	Who Creates It	Purpose
PersistentVolume (PV)	Cluster admin (or auto)	A piece of storage in the cluster
PersistentVolumeClaim (PVC)	App developer	A request: 'I need X GB of storage'
StorageClass (SC)	Cluster admin	A template for auto-creating PVs on demand

Real-World Analogy

StorageClass = the storage menu (gp3 SSD, standard HDD, NFS share). **PVC** = your order ("I want 10 GB of gp3"). **PV** = the actual storage allocated to fulfill your order. The kitchen (StorageClass) creates PVs dynamically based on PVC orders.

Access Modes — Who Can Read/Write?

Mode	Short	Meaning
ReadWriteOnce	RWO	Mounted by a single node (most common — EBS, local disk)
ReadOnlyMany	ROX	Mounted read-only by many nodes
ReadWriteMany	RWX	Mounted read-write by many nodes (NFS, EFS)
ReadWriteOncePod	RWOP	Mounted by exactly one pod (newer, stricter than RWO)

PVC YAML — How Apps Request Storage

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-data-pvc
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi          # I want 5 GB
  storageClassName: standard # which SC to use
```

Mount the PVC into a Pod

```
spec:
  containers:
    - name: my-app
      image: nginx
      volumeMounts:
        - mountPath: /data      # path inside container
          name: my-storage
  volumes:
    - name: my-storage
      persistentVolumeClaim:
        claimName: my-data-pvc # link to the PVC above
```

StorageClass — The Storage Template

When you create a PVC referencing a StorageClass, Kubernetes auto-provisions a PV that matches your request. This is **dynamic provisioning** — the modern approach.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
  fsType: ext4
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

Reclaim Policies — What Happens When PVC Is Deleted

Policy	Behavior	Use Case
Delete	PV and underlying storage are deleted	Dev/test (default for cloud)
Retain	PV kept; data preserved for manual recovery	Production databases

Useful Commands

```
kubectl get pv          # see all PersistentVolumes
kubectl get pvc         # see your claims
kubectl get storageclass # see available templates
kubectl describe pvc my-data-pvc # debug binding issues
```

Always use dynamic provisioning via StorageClass in real clusters. Manually creating PVs is only for learning the binding mechanic.

4. StatefulSets — Stateful Workloads

When pods need stable identities (databases, queues, distributed systems)

The Problem with Deployments for Stateful Apps

Deployments treat pods as **interchangeable**. Each pod gets a random name (my-app-7d8f4b6c-x9k2p) and any pod can replace any other. That's fine for stateless apps but breaks databases:

- A MySQL replica needs to know *which* primary to follow — it needs a stable hostname.
- Storage attached to mysql-0 must come back to mysql-0 on restart, not to mysql-1.
- Replicas must come up in order (primary first, then replicas).

What StatefulSets Provide

Property	Deployment	StatefulSet
Pod names	Random suffix	Predictable: mysql-0, mysql-1, mysql-2
Pod identity	Interchangeable	Stable network identity per pod
Storage	Shared or none	Each pod gets its own PVC (preserved)
Startup order	All in parallel	Ordered: 0 first, then 1, then 2
Use case	Stateless apps	Databases, queues, distributed systems

Headless Service — The Missing Piece

StatefulSets need a **headless Service** (clusterIP: None) to give each pod a stable DNS name like mysql-0.mysql.default.svc.cluster.local.

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None      # ← THIS makes it headless
  selector:
    app: mysql
  ports:
    - port: 3306
```

StatefulSet YAML

```
apiVersion: apps/v1
kind: StatefulSet
```

```

metadata:
  name: mysql
spec:
  serviceName: mysql          # links to the headless service
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
      volumeClaimTemplates:    # auto-creates PVC per pod
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 10Gi

```

How It Plays Out

When you apply this manifest, Kubernetes creates:

- Pods named `mysql-0`, `mysql-1`, `mysql-2` (in that order)
- PVCs named `data-mysql-0`, `data-mysql-1`, `data-mysql-2`
- DNS names: `mysql-0.mysql`, `mysql-1.mysql`, `mysql-2.mysql`

Useful Commands

```

kubectl get statefulset
kubectl get pvc                # see auto-created PVCs
kubectl scale sts mysql --replicas=5  # scale up (creates 3, 4)
kubectl delete pod mysql-1          # watch K8s recreate as mysql-1

```

Never use a Deployment for a database. The stable hostname `mysql-0.mysql` is what allows replicas to always find the primary. Deployments cannot give you this.

5. Jobs & CronJobs — One-Off and Scheduled Tasks

Running batch workloads that complete and exit

Why You Need Them

Deployments and StatefulSets run **forever**. But many real workloads should **run, finish, and exit**: database migrations, nightly backups, reports, image processing batches.

Job — Run Once Until Completion

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  completions: 1           # run once successfully
  parallelism: 1           # one pod at a time
  backoffLimit: 4          # retry up to 4 times on failure
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: migrate
        image: my-app-migrator:v1
        command: ["python", "migrate.py"]
```

completions: total successful pods needed.

parallelism: how many pods can run at the same time.

backoffLimit: how many retries before giving up.

CronJob — Run on a Schedule

A CronJob creates a Job at scheduled times using standard cron syntax.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"      # 2 AM every day (cron syntax)
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
```

```
containers:
- name: backup
  image: my-backup-tool:v1
  command: ["sh", "-c", "backup.sh"]
```

Cron Syntax Refresher

Schedule	Meaning
* * * * *	Every minute
0 * * * *	Every hour at minute 0
0 2 * * *	Every day at 2:00 AM
*/15 * * * *	Every 15 minutes
0 0 * * 0	Every Sunday at midnight

Useful Commands

```
kubectl get jobs
kubectl get cronjobs
kubectl logs job/db-migration          # see job output
kubectl create job --from=cronjob/nightly-backup manual-backup
                                         # trigger a CronJob now
```

6. DaemonSets — One Pod Per Node

Running a copy of a pod on every node automatically

Why DaemonSets Exist

Some workloads need to run on EVERY node — log collectors, monitoring agents, network plugins. You can't use a Deployment because the scheduler might place all replicas on one node. DaemonSets guarantee one pod per node.

Common Real-World DaemonSets

DaemonSet	Purpose
Fluentd / Filebeat	Log collection from each node
Prometheus node-exporter	Metrics from each node's hardware
Calico / Flannel	Network plugin (CNI)
Datadog / New Relic agent	Monitoring
AWS VPC CNI (aws-node)	EKS pod networking

DaemonSet YAML

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
spec:
  selector:
    matchLabels:
      app: log-agent
  template:
    metadata:
      labels:
        app: log-agent
    spec:
      containers:
        - name: agent
          image: fluentd:v1.16
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
```

```
path: /var/log
```

Notice: No replicas field — DaemonSets count nodes, not replicas. Add a node, get a pod automatically. Remove a node, the pod goes with it.

Useful Commands

```
kubectl get daemonset -A
kubectl get ds log-agent
kubectl describe ds log-agent
```

7. Init Containers & Sidecar Pattern

Composing pods from multiple containers for real-world needs

Init Containers — Pre-Flight Setup

Init containers run BEFORE your main app container starts. They run in order, one at a time, and must each succeed before the next runs. The main container starts only after ALL init containers complete.

Common Use Cases

- Wait for a dependency to be ready (e.g. wait for the database)
- Download config files or git repos
- Set permissions on volumes
- Run database migrations before app starts

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command: ['sh', '-c', 'until nc -z mysql 3306; do echo waiting; sleep 2; done']
    - name: run-migrations
      image: my-app-migrator:v1
      command: ['python', 'migrate.py']
  containers:
    - name: my-app
      image: my-app:v1
      # only starts AFTER both init containers complete
```

Sidecar Pattern — Helper Container

A sidecar runs ALONGSIDE your main container, sharing the same network and volumes. It provides cross-cutting features (logging, proxying, syncing) without modifying your main app.

Common Sidecars in Production

Sidecar	Job
Envoy / Istio proxy	Service mesh — traffic routing, mTLS
Fluent Bit	Tail logs from main app, ship to Loki/ES
Cloud SQL Auth Proxy	Secure DB connection
Vault Agent	Inject secrets into shared volume

```
spec:
  containers:
    - name: my-app
```

```
image: my-app:v1
volumeMounts:
- name: shared-logs
  mountPath: /var/log/app
- name: log-shipper          # ← sidecar
  image: fluent/fluent-bit:latest
  volumeMounts:
  - name: shared-logs
    mountPath: /var/log/app
volumes:
- name: shared-logs
  emptyDir: {}
```

Mental model: Init containers happen in *sequence* before main. Sidecars run in *parallel* with main. Both share the pod's network and volumes.

8. Advanced Ingress — TLS, Path Routing, Controllers

Going beyond Phase 1's intro to real production routing

Phase 1 Recap + What's Next

In Phase 1 you learned *what* Ingress is. Phase 2 covers *HOW* to actually make it work: installing a controller, real path/host routing, and TLS.

Critical: The Ingress resource does NOTHING by itself. It's just a set of rules. You need an **Ingress Controller** (NGINX, Traefik, AWS ALB) to actually read those rules and route traffic. No controller = no routing.

Installing NGINX Ingress Controller

```
# On kind / minikube / bare-metal
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml

# Verify it's running
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Path-Based Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: myapp.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 8080
          - path: /web
            pathType: Prefix
            backend:
              service:
                name: web-service
```

```
port:
  number: 80
```

Host-Based Routing (Multiple Apps, One Cluster)

```
spec:
  ingressClassName: nginx
  rules:
  - host: api.mycompany.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service
            port: { number: 8080 }
  - host: shop.mycompany.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: shop-service
            port: { number: 80 }
```

TLS / HTTPS

To serve HTTPS, you need a TLS Secret containing your certificate and key. In production, use **cert-manager** + Let's Encrypt for automatic, free certificates that auto-renew.

```
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - api.mycompany.com
    secretName: api-tls-cert      # secret holding cert + key
  rules:
  - host: api.mycompany.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service
            port: { number: 8080 }
```

pathType — A Common Source of Bugs

pathType	Behavior	Example
Prefix	Matches path AND any sub-paths	/api matches /api, /api/users

Exact	Matches the path exactly only	/api matches only /api
ImplementationSpecific	Up to the controller	Avoid — be explicit

9. Horizontal Pod Autoscaler (HPA)

Auto-scaling pods based on load — no human intervention

What HPA Does

HPA automatically increases or decreases the number of pod replicas based on observed metrics (typically CPU or memory). Your traffic doubles? HPA doubles your pods. Traffic drops? It scales back down to save resources.

Prerequisite — metrics-server

HPA needs the metrics-server installed in your cluster. Without it, you'll see <unknown> in HPA status and nothing scales.

```
# On minikube
minikube addons enable metrics-server

# On kind / bare-metal
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# Verify
kubectl top nodes
kubectl top pods
```

HPA YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70    # scale when avg CPU > 70%
```

Imperative Shortcut

```
kubectl autoscale deployment my-app --cpu-percent=70 --min=2 --max=10
```

How HPA Decides

Every 15 seconds (default), HPA checks the metric and adjusts replicas. The math is simple:

$\text{desired replicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$

Example: 4 pods, current CPU 90%, target 60% → desired = $\text{ceil}(4 \times 90 / 60) = 6$ pods.

Useful Commands

```
kubectl get hpa
kubectl describe hpa my-app-hpa      # see scaling decisions
kubectl get hpa --watch              # live updates
```

Important: HPA needs your pods to have CPU *requests* set. Without them, the percentage calculation has nothing to compare against. Always set requests on autoscaled deployments.

10. RBAC & ServiceAccounts

Controlling who and what can do what in your cluster

The Problem

By default in many clusters, every workload runs as the **default** ServiceAccount with broad permissions. If your app is compromised, the attacker can talk to the K8s API and do real damage. RBAC stops this.

The Four Pieces of RBAC

Object	Purpose
Role	List of allowed actions, scoped to ONE namespace
ClusterRole	Same as Role, but cluster-wide (all namespaces)
RoleBinding	Attaches a Role to a user/group/ServiceAccount
ClusterRoleBinding	Attaches a ClusterRole cluster-wide

ServiceAccount — Identity for Pods

Every pod runs as a ServiceAccount. When your app calls the K8s API (e.g. an operator listing pods), it authenticates as that SA. RBAC controls what that SA can do.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-runner
  namespace: dev
```

Role — What Actions Are Allowed

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

RoleBinding — Glue Them Together

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
```

```

metadata:
  name: read-pods-binding
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-runner
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

Common Verbs

Verb	Allows
get	Read a single resource
list	Read all resources of a kind
watch	Subscribe to changes
create	Create new resources
update / patch	Modify existing resources
delete	Remove resources

Make a Pod Use Your ServiceAccount

```

spec:
  serviceAccountName: app-runner    # ← link the SA to the pod
  containers:
  - name: my-app
    image: my-app:v1

```

Test Permissions Like a Pro

```

# As yourself
kubectl auth can-i list pods -n dev

# As another user/SA
kubectl auth can-i list pods --as=system:serviceaccount:dev:app-runner -n dev

# What CAN this SA do?
kubectl auth can-i --list --as=system:serviceaccount:dev:app-runner

```

Production rule: Always create a dedicated ServiceAccount per app. Never use default — it can inherit permissions you didn't intend.

11. NetworkPolicies — Pod-Level Firewall

By default, all pods can talk to all pods. NetworkPolicies fix that.

Default Behavior — Wide Open

Out of the box, Kubernetes lets every pod talk to every other pod, in any namespace, on any port. If your frontend pod is compromised, the attacker can reach the database pod directly.

NetworkPolicies restrict this.

Important: NetworkPolicies require a CNI plugin that supports them (Calico, Cilium, Weave). On vanilla minikube/kind, they may not be enforced until you install one.

Default-Deny — The Right Starting Point

Best practice: deny everything by default in a namespace, then explicitly allow what's needed.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: prod
spec:
  podSelector: {}          # apply to all pods in prod
  policyTypes:
    - Ingress
    - Egress
```

Allow Only Specific Traffic

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow-from-frontend
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend          # only frontend pods can reach api
      ports:
        - protocol: TCP
          port: 8080
```

What You Can Match On

Selector	Targets
podSelector	Pods with specific labels
namespaceSelector	All pods in matching namespaces
ipBlock	External CIDR ranges (e.g. office IP)

Pattern that works: Start with default-deny in production namespaces. Add explicit allow rules ONLY for traffic you've verified is needed. This is the right security posture from day one.

12. Pod Scheduling — nodeSelector, Affinity, Taints

Controlling WHERE your pods run

Why Scheduling Matters

By default the scheduler picks any node with enough capacity. That's fine until you need: GPU nodes only for ML workloads, all replicas spread across availability zones for HA, dedicated nodes for sensitive workloads.

nodeSelector — The Simple Way

Label your nodes, then tell pods to land only on nodes with that label.

```
# Label a node
kubectl label nodes node-1 disktype=ssd

# Pod that demands SSD
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: my-app
    image: my-app:v1
```

Node Affinity — The Flexible Way

Like nodeSelector but with operators (In, NotIn, Exists). You can also express **preferences** instead of hard requirements.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:  # MUST
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values: ["ssd", "nvme"]
      preferredDuringSchedulingIgnoredDuringExecution:  # PREFER
        - weight: 1
          preference:
            matchExpressions:
            - key: zone
              operator: In
              values: ["us-east-1a"]
```

Pod Anti-Affinity — Spread Replicas Across Nodes

Critical for HA: don't let all 3 replicas land on the same node — one node outage would kill your service.

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["my-app"]
          topologyKey: "kubernetes.io/hostname"
        # ↑ no two my-app pods on the same node
```

Taints & Tolerations — Reserve Nodes

Taints repel pods from nodes. **Tolerations** let specific pods land on tainted nodes. Use this to reserve expensive nodes (GPUs) for specific workloads.

```
# Taint a GPU node — repel all pods by default
kubectl taint nodes gpu-node-1 dedicated=gpu:NoSchedule

# Allow only ML pods to tolerate this taint
spec:
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "gpu"
      effect: "NoSchedule"
  containers:
    - name: ml-worker
      image: tensorflow:latest
```

Taint Effects

Effect	Meaning
NoSchedule	Don't schedule new pods unless they tolerate
PreferNoSchedule	Try to avoid scheduling, but allow if needed
NoExecute	Evict existing pods that don't tolerate

13. Advanced kubectl — jsonpath, custom-columns

Going beyond 'kubectl get pods' to ninja-level workflows

Why This Matters

Basic kubectl gets you started. Advanced kubectl makes you fast. Senior engineers debug production issues in seconds because they know these tricks.

Output Formats Cheat Sheet

```
kubectl get pods -o wide           # extra columns (node, IP)
kubectl get pods -o yaml          # full YAML
kubectl get pods -o json          # full JSON
kubectl get pods -o name          # just names
kubectl get pods -o jsonpath='...' # custom JSON queries
kubectl get pods -o custom-columns=...# custom table
```

jsonpath — Extract Specific Fields

```
# All pod names
kubectl get pods -o jsonpath='{.items[*].metadata.name}'

# All container images across all pods
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'

# Pod status of one pod
kubectl get pod my-pod -o jsonpath='{.status.phase}'

# All nodes and their kubelet versions
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{ "\t" }{.status.nodeInfo.kubeletVersion}{ "\n" }{end}'
```

Custom Columns — Tables Your Way

```
kubectl get pods -o custom-columns=\
  NAME:.metadata.name,\
  STATUS:.status.phase,\
  NODE:.spec.nodeName,\
  IMAGE:.spec.containers[0].image
```

Filter and Sort

```
# Sort by creation time
kubectl get pods --sort-by=.metadata.creationTimestamp

# Filter with field-selector
kubectl get pods --field-selector status.phase=Running

# Filter with label-selector
```

```
kubectl get pods -l app=my-app  
kubectl get pods -l 'env in (prod,staging)'
```

Generate YAML Skeletons (CKA Exam Hack)

```
alias k=kubectl  
export do="--dry-run=client -o yaml"  
  
# Generate a pod skeleton  
k run nginx --image=nginx $do > pod.yaml  
  
# Generate a deployment skeleton  
k create deploy web --image=nginx --replicas=3 $do > deploy.yaml  
  
# Generate a service skeleton  
k expose deploy web --port=80 $do > svc.yaml
```

Diff Before You Apply

```
kubectl diff -f manifest.yaml      # show what would change
```

Install these tools immediately via krew (kubectl plugin manager):

- kubectx — switch contexts fast
- kubens — switch namespaces fast
- stern — tail logs from multiple pods
- k9s — full terminal UI for K8s

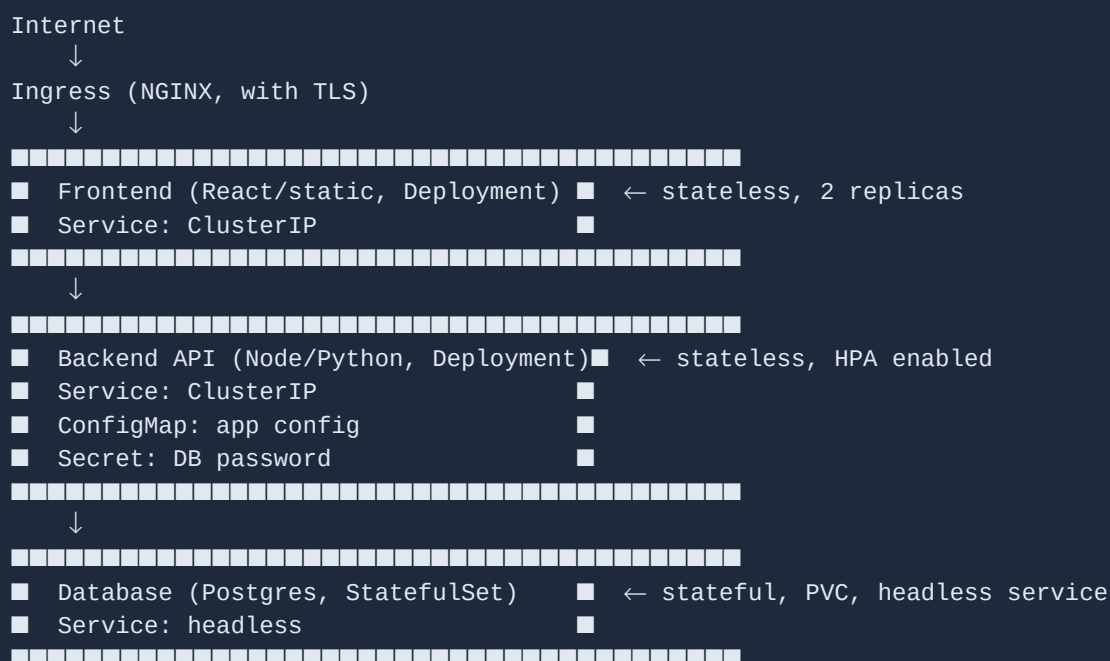
14. Phase 2 Project — 3-Tier App on K8s

Apply everything you've learned in one real project

Project: Build a 3-Tier Web App

Build a complete app that exercises every Phase 2 concept. Don't skip this — this is what turns theory into permanent skill.

Architecture



Requirements Checklist

Your project must include all of these:

- Three Deployments (frontend, backend) + StatefulSet (postgres)
- ClusterIP Services for frontend & backend
- Headless Service for postgres
- PersistentVolumeClaim for postgres data (5+ GB)
- ConfigMap for app config (DB host, port, env)
- Secret for DB password (base64)
- Liveness + Readiness probes on every container
- Resource requests + limits on every container
- HPA on the backend (CPU > 70% scales 2 → 5)
- Init container in backend that waits for postgres to be ready

- Sidecar in backend for log shipping (or just demonstrate the pattern)
- Ingress with path routing: `/api` → backend, `/` → frontend
- TLS secret on the Ingress (self-signed for local is fine)
- Dedicated ServiceAccount for backend with RBAC limited to read-only
- NetworkPolicy: only backend can reach postgres
- Run everything in a custom namespace, not default

Suggested Approach

1. Build it on your local **kind** cluster first.
2. Write each YAML separately, apply one at a time, verify.
3. Push your YAMLs to a GitHub repo — this becomes the input for Phase 3 (Helm + GitOps).
4. Document what broke and how you fixed it. This becomes your interview-talking-points list.

You'll get stuck. That's the point. Every error you debug builds real skill. Read the events (`kubectl describe`) and the logs (`kubectl logs`) before searching online.

15. Phase 2 Summary Cheatsheet

One-page reference for everything in Phase 2

New Resources You Now Know

Resource	What It Does
Probe	Health check: liveness, readiness, startup
PV / PVC / StorageClass	Persistent storage layer
StatefulSet	Pods with stable identity & per-pod storage
Job / CronJob	Run-to-completion and scheduled tasks
DaemonSet	One pod per node (agents)
Init Container	Pre-flight setup before main container
Sidecar	Helper container alongside main
HPA	Auto-scale pods on CPU/memory
Role / RoleBinding	Namespace-scoped permissions
ClusterRole / ClusterRoleBinding	Cluster-wide permissions
ServiceAccount	Identity for pods & API auth
NetworkPolicy	Pod-level firewall rules

Most Used Phase 2 Commands

```
# Probes / health
kubectl describe pod <name> | grep -A 5 "Liveness\|Readiness"

# Resource usage
kubectl top nodes
kubectl top pods --sort-by=memory

# Storage
kubectl get pv,pvc
kubectl get storageclass
```



```

# StatefulSets
kubectl get sts
kubectl scale sts <name> --replicas=N

# Jobs / CronJobs
kubectl get jobs,cronjobs
kubectl create job --from=cronjob/<name> manual-run

# DaemonSets
kubectl get ds -A

# Autoscaling
kubectl autoscale deploy <name> --cpu-percent=70 --min=2 --max=10
kubectl get hpa --watch

# RBAC debugging
kubectl auth can-i <verb> <resource> --as=<user|sa>

# Scheduling
kubectl label nodes <node> key=value
kubectl taint nodes <node> key=value:NoSchedule

# Advanced inspection
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase
kubectl diff -f manifest.yaml

```

What Comes Next — Phase 3 Preview

You can now run real production-style apps on Kubernetes. Phase 3 will automate everything you just did manually:

- **Helm** — package your manifests as reusable, parameterized charts
- **Kustomize** — environment-specific overlays without templates
- **ArgoCD / FluxCD** — GitOps: Git is the single source of truth
- **GitHub Actions** — CI pipeline that builds, tests, and triggers deploy
- **Image promotion strategy** — dev → staging → prod with proper gates

You have completed Phase 2. You now know how to run real, production-grade Kubernetes workloads — with health, identity, storage, scaling, and security. Build the project, push it to GitHub, and prepare for Phase 3.